

**ACCESS - ADAPTIVE COGNITIVE & EDUCATIONAL
SKILL SUPPORT PLATFORM**

MUHAMMAD ALIFF BIN AFFANDI

UNIVERSITI TEKNIKAL MALAYSIA MELAKA

ACCESS - ADAPTIVE COGNITIVE & EDUCATIONAL SKILL SUPPORT
PLATFORM

MUHAMMAD ALIFF BIN AFFANDI

This report is submitted in partial fulfillment of the requirements for the
Bachelor of Computer Science (Software Development) with Honours.

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY
UNIVERSITI TEKNIKAL MALAYSIA MELAKA

2026

DECLARATION

I hereby declare that this project entitled
ACCESS - ADAPTIVE COGNITIVE & EDUCATIONAL SKILL SUPPORT
PLATFORM
is written by me and is my own effort and that no part has been plagiarized without
citations.

STUDENT : _____ Date : _____
(MUHAMMAD ALIFF BIN AFFANDI)

I hereby declare that I have read this project report and found this project report is
sufficient in term of the scope and quality for the award of Bachelor of Computer
Science (Software Development) with Honours.

SUPERVISOR : _____ Date : _____
(TS. DR. ZURAINI BINTI OTHMAN)

DEDICATION

To my beloved parents, whose endless sacrifices and unwavering faith have been my foundation through every challenge you faced to give me the opportunity to pursue my dreams. Your love and encouragement carried me through the most difficult moments of this journey.

To Ts. Dr. Zuraini Binti Othman, whose mentorship, patience, and invaluable guidance shaped not only this research but also my growth as a researcher and software developer. Your dedication to accessibility and inclusive design inspired the very core of this project.

To all neurodivergent learners who navigate a world not yet built for them, this work is dedicated to you. May ACESS serve as a small step toward a future where educational platforms adapt to every mind, rather than demanding that every mind adapt to them.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor, Ts. Dr. Zuraini Binti Othman, for her continuous guidance, patience, and insightful feedback throughout the development of this project. Her expertise in software engineering and dedication to accessibility has been instrumental in shaping the ACCESS platform into a functional and meaningful system.

I also extend my sincere appreciation to the Faculty of Information and Communication Technology (FICT) at Universiti Teknikal Malaysia Melaka for providing the necessary resources, laboratory facilities, and learning environment that made this research possible. The knowledge imparted throughout my studies formed the bedrock upon which this project was built.

My heartfelt thanks go to my family and friends, who offered unwavering emotional and moral support during the challenging moments of this journey. Their encouragement was a constant source of motivation during late nights and difficult debugging sessions.

Finally, I am grateful to all the educators and learners who participated in the user acceptance testing. Their candid feedback was vital in refining the platform's accessibility features and ensuring that ACCESS truly serves its intended purpose.

ABSTRACT

Traditional e-learning platforms are predominantly designed with a one-size-fits-all approach, presenting significant accessibility barriers for neurodivergent learners, particularly those with Dyslexia, Attention Deficit Hyperactivity Disorder (ADHD), and Autism Spectrum Disorder (ASD). These systems typically feature dense typography, complex navigational structures, and rigid content delivery that contribute to cognitive overload and high dropout rates. This project presents the development of the Adaptive Cognitive and Educational Skill Support (ACESS) platform, a web-based learning management system engineered to dynamically accommodate diverse cognitive profiles through natively integrated accessibility features. The system was developed using the Agile Scrum methodology over six iterative sprints, employing Next.js with the App Router for server-rendered React components, Supabase for backend-as-a-service with PostgreSQL Row-Level Security, and Tailwind CSS v4 for dynamic theme injection. ACESS implements core features including a rule-based adaptive engine that automatically recommends remedial content based on assessment performance, configurable accessibility profiles that alter typography, spacing, and motion preferences in real time via CSS custom properties, a strict Focus Mode for learners with attention-related challenges, and comprehensive analytics dashboards for educators to monitor engagement and identify at-risk students. Evaluation through user acceptance testing demonstrated measurable improvements in task completion rates and user satisfaction scores among neurodivergent participants compared to baseline interactions with conventional platforms. The findings indicate that a purpose-built, architecturally adaptive system can significantly reduce cognitive friction and promote equitable educational outcomes for learners with diverse neurological needs.

ABSTRAK

Platform e-pembelajaran tradisional kebanyakannya direka dengan pendekatan satu-saiz-untuk-semua, yang menimbulkan halangan kebolehcapaian yang signifikan bagi pelajar neurodivergen, terutamanya mereka yang mengalami Disleksia, Attention Deficit Hyperactivity Disorder (ADHD), dan Autism Spectrum Disorder (ASD). Sistem-sistem ini biasanya menampilkan tipografi yang padat, struktur navigasi yang kompleks, dan penyampaian kandungan yang tegar, yang menyumbang kepada beban kognitif dan kadar keciciran yang tinggi. Projek ini membentangkan pembangunan platform Adaptive Cognitive and Educational Skill Support (ACESS), sebuah sistem pengurusan pembelajaran berasaskan web yang direka untuk menampung profil kognitif yang pelbagai secara dinamik melalui ciri kebolehcapaian yang diintegrasikan secara natif. Sistem ini dibangunkan menggunakan metodologi Agile Scrum melalui enam lelaran sprint, menggunakan Next.js dengan App Router untuk komponen React yang dipaparkan di pelayan, Supabase sebagai backend-as-a-service dengan keselamatan peringkat baris PostgreSQL, dan Tailwind CSS v4 untuk suntikan tema dinamik. ACESS melaksanakan ciri teras termasuk enjin adaptif berasaskan peraturan yang secara automatik mengesyorkan kandungan pemulihan berdasarkan prestasi penilaian, profil kebolehcapaian boleh konfigurasi yang mengubah tipografi, jarak, dan keutamaan gerakan secara masa nyata melalui pemboleh ubah CSS, mod Fokus yang ketat untuk pelajar yang mempunyai cabaran berkaitan perhatian, serta papan pemuka analitik komprehensif untuk pendidik memantau penglibatan dan mengenal pasti pelajar berisiko. Penilaian melalui ujian penerimaan pengguna menunjukkan peningkatan yang boleh diukur dalam kadar penyelesaian tugas dan skor kepuasan pengguna dalam kalangan peserta neurodivergen berbanding interaksi asas dengan platform konvensional. Dapatan kajian menunjukkan bahawa sistem yang dibina secara adaptif dari segi seni bina dapat mengurangkan geseran kognitif dengan ketara dan menggalakkan hasil pendidikan yang saksama untuk pelajar dengan keperluan neurologi yang pelbagai.

TABLE OF CONTENTS

	PAGE
DECLARATION	ii
DEDICATION	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
ABSTRAK	vi
TABLE OF CONTENTS	vii
LIST OF TABLES	xi
LIST OF FIGURES	xii
LIST OF ABBREVIATIONS	xiii
CHAPTER 1: INTRODUCTION	1
1.1 Background of Study	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope	3
1.4.1 Target Users	3
1.4.2 Core System Modules	3
1.5 Significance of Project	5
1.6 Expected Output	5
1.7 Conclusion	7
CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY	8
2.1 Introduction	8
2.2 Theoretical Frameworks for Cognitive Accessibility	8
2.2.1 Universal Design for Learning (UDL)	8
2.2.2 Web Content Accessibility Guidelines (WCAG) 2.2	9

2.3	Neurodivergent Profiles and Interface Interventions	10
2.3.1	Attention Deficit Hyperactivity Disorder (ADHD)	10
2.3.2	Dyslexia	10
2.3.3	Autism Spectrum Disorder (ASD)	11
2.4	Comparative Analysis of Existing LMS Platforms	11
2.4.1	Moodle	11
2.4.2	Canvas LMS	12
2.4.3	Blackboard Learn	12
2.4.4	Google Classroom	12
2.5	Comparison Matrix	13
2.5.1	Alternative Approaches and Justification	13
2.6	Project Methodology	14
2.7	Project Requirements	15
2.7.1	Software Requirements	15
2.7.2	Hardware Requirements	17
2.7.3	Other Requirements	18
2.8	Project Schedule and Milestones	18
2.9	Summary	20
CHAPTER 3: SYSTEM ANALYSIS		21
3.1	Introduction	21
3.2	Problem Analysis	21
3.2.1	Current System Scenario	21
3.2.2	Detailed Problem Breakdown	22
3.3	Requirement Analysis	23
3.3.1	Data Requirement	23
3.3.2	Functional Requirement	24
3.3.3	Non-Functional Requirement	28
3.3.4	Others Requirement	30
3.4	System Architecture	31
3.4.1	Context Diagram	31
3.4.2	System Architecture Diagram	33
3.4.3	Deployment Diagram	34

3.5	Module Design	35
3.6	Summary	35
CHAPTER 4: SYSTEM DESIGN		37
4.1	Introduction	37
4.2	Behavioral Design (Activity Diagrams)	37
4.2.1	Student Activity Diagram	38
4.2.2	Educator Activity Diagram	39
4.2.3	Administrator Activity Diagram	40
4.3	Interaction Design (Sequence Diagrams)	40
4.3.1	Sequence: Course Enrollment	41
4.3.2	Sequence: Lesson Learning & Telemetry	42
4.3.3	Sequence: Quiz Attempt & Adaptive Recommendation	43
4.3.4	Sequence: Course Creation & Media Upload	44
4.4	Database Design & Entity Relationship	44
4.5	Comprehensive Data Dictionary	45
4.5.1	Core Identity and Configuration Tables	45
4.5.2	Curriculum Structure Tables	47
4.5.3	Telemetry and Assessment Tables	48
4.5.4	Specialized Feature Tables	51
4.6	Interface Design and Implementation Mechanics	52
4.6.1	Student Interface Design (Focus & Adaptability)	52
4.6.2	Educator Interface Design (Creation & Analytics)	53
4.6.3	Administrator Interface Design (Governance)	53
4.7	Systemic Implementation Details	54
4.7.1	Interactive Video Questions	54
4.7.2	Database Trigger Architecture (Notifications)	54
4.8	Summary	54
References		56

LIST OF TABLES

	PAGE
Table 2.1 LMS Feature Comparison Matrix	13
Table 2.2 Project Schedule and Milestones	19
Table 3.1 Functional Requirements: Student Module	26
Table 3.2 Functional Requirements: Educator Module	27
Table 3.3 Functional Requirements: Admin Module	27

LIST OF FIGURES

	PAGE
Figure 2.1 Software tools used in ACESS development	17
Figure 2.2 Gantt chart of ACESS development sprint schedule	19
Figure 3.1 Placeholder for Current System Problem Analysis Diagram	22
Figure 3.2 Placeholder for Use Case Diagram	25
Figure 3.3 Placeholder for Context Diagram	32
Figure 3.4 Placeholder for System Architecture Diagram	33
Figure 3.5 Placeholder for Deployment Diagram	34
Figure 4.1 Placeholder for Student Activity Diagram	38
Figure 4.2 Placeholder for Educator Activity Diagram	39
Figure 4.3 Placeholder for Administrator Activity Diagram	40
Figure 4.4 Placeholder for Course Enrollment Sequence Diagram	41
Figure 4.5 Placeholder for Lesson Learning Sequence Diagram	42
Figure 4.6 Placeholder for Quiz Attempt Sequence Diagram	43
Figure 4.7 Placeholder for Course Creation Sequence Diagram	44
Figure 4.8 Placeholder for Database ERD	45

LIST OF ABBREVIATIONS

ACCESS	- Adaptive Cognitive & Educational Skill Support
ADHD	- Attention Deficit Hyperactivity Disorder
ERD	- Entity Relationship Diagram
LMS	- Learning Management System
OOAD	- Object-Oriented Analysis and Design
PSM	- Projek Sarjana Muda
RBAC	- Role-Based Access Control
TTS	- Text-to-Speech
UDL	- Universal Design for Learning
WCAG	- Web Content Accessibility Guidelines

CHAPTER 1: INTRODUCTION

1.1 Background of Study

In the current educational landscape, instruction is delivered through a blend of traditional face-to-face classroom settings and online platforms. While digital tools have become increasingly common to support learning, many institutions still rely heavily on traditional teaching methods. This hybrid approach presents unique challenges for neurodivergent individuals, specifically those with Dyslexia, Attention Deficit Hyperactivity Disorder (ADHD), and Autism Spectrum Disorder (ASD). Whether learning occurs in a physical classroom or online, the tools used to deliver content often lack the flexibility required to accommodate diverse cognitive profiles.

Existing Learning Management Systems (LMS) are typically designed with a “one-size-fits-all” approach. They present dense textual information, complex navigation structures, and rigid assessment formats. These standard interfaces can erect significant barriers for neurodivergent learners. For instance, learners with Dyslexia may struggle with standard typography and line spacing, while those with ADHD might find cluttered interfaces and unstructured learning paths overwhelming.

Therefore, there is a critical need for a natively adaptive learning platform that can seamlessly support both face-to-face and remote learning environments. The Adaptive Cognitive Education Support System (ACESS) is proposed to bridge this gap. ACESS is engineered to provide an inclusive digital learning environment that dynamically alters its interface and content delivery based on the specific cognitive requirements of the user. By integrating features such as dynamic typography adjustment, focus-driven layouts, and rule-based adaptive learning paths, ACESS aims to ensure that educational materials are

accessible and equitable for all learners, regardless of the instructional setting.

1.2 Problem Statement

Despite the availability of various educational technologies, significant cognitive accessibility barriers persist. The core problem this project addresses is the systemic inadequacy of current educational platforms to dynamically support the specific neurological requirements of learners with Dyslexia, ADHD, and ASD.

Existing platforms often rely on static content delivery. They require neurodivergent learners to manually seek out third-party browser extensions or external text-to-speech tools to consume content. This forces the burden of accessibility onto the learner. Furthermore, when learners struggle with specific topics, traditional systems lack the heuristic intelligence to automatically intervene and restructure the learning path.

Specifically, the problems identified are:

1. Standard educational platforms utilize rigid, non-adjustable typography and dense layouts that induce visual crowding and reading fatigue for learners with Dyslexia.
2. Current interfaces present excessive cognitive load through complex navigation and unstructured content delivery, which overwhelms learners with ADHD and executive dysfunction.
3. Existing systems lack the capability to autonomously identify knowledge gaps during assessments and dynamically route learners to required remedial materials.

1.3 Objectives

The primary goal of this project is to develop an inclusive Learning Management System that addresses the cognitive barriers faced by neurodivergent learners. The specific objectives are:

- To design and develop an adaptive learning management system (ACCESS) that integrates dynamic interface modifications, including dyslexia-friendly typography and focus-mode layouts.
- To implement a rule-based adaptive engine that monitors learner assessment scores and automatically recommends specific prerequisite lessons to address identified knowledge gaps.
- To evaluate the effectiveness of the system in providing an accessible and structured learning experience for users with diverse cognitive profiles.

1.4 Scope

The scope of the ACCESS platform defines the boundaries of the system architecture, target users, and core functional modules. The system is designed to act as a supportive educational tool for both traditional and online learning environments.

1.4.1 Target Users

- **Learners:** The primary consumers of the educational content. The interface is specifically optimized for neurodivergent individuals, though it remains highly beneficial for neurotypical users.
- **Educators:** The course creators and administrators who require structured, intuitive tools to author accessible content, manage media assets, and monitor student progress.
- **Administrators:** System operators responsible for platform governance, user role management, and global accessibility configuration.

1.4.2 Core System Modules

The ACCESS architecture is divided into the following core modules:

- **Authentication and Profile Module:** Manages secure registration, login, role-based access control (learner, educator, admin), email verification, password reset, session timeout, and profile management.
- **Accessibility and Personalization Engine:** The central differentiator of ACCESS. Stores per-user `user_accessibility_settings` defining cognitive profiles (Dyslexia, ADHD, Autism, Dyscalculia, Universal) with granular controls over font, spacing, theme, tint, reading spotlight, focus mode, reduced motion, simplified UI, TTS, captions, and keyboard navigation. Settings are applied globally via CSS data attributes on the root element, requiring no manual configuration after onboarding.
- **Educator Authoring Module:** Provides an interface for creating structured courses with chapters and milestones, writing lessons via Tiptap, uploading video assets with time-coded questions, designing interactive activities (flashcards, drag-and-drop, fill-in-the-blanks, memory games, timelines, H5P), managing lesson version history, and configuring quiz, certificate, and achievement criteria.
- **Adaptive Learning Module:** Governs course enrollment, lesson progression, dynamic UI alterations based on accessibility profiles, and personalized recommendations triggered by quiz performance – including a rule-based engine that suggests revision, standard, advanced, or cross-course content.
- **Interactive Assessment Module:** Handles timed and untimed quiz delivery, multiple attempts, automated grading, answer review, and pass/fail threshold evaluation that triggers remedial recommendations when scores fall below configured thresholds.
- **Analytics and Telemetry Module:** Tracks engagement metrics (time per lesson, completion rates, scores, enrollment trends) and provides educators with per-student dashboards with at-risk detection, plus platform-wide analytics and custom report generation for administrators.
- **Certificate and Achievement Module:** Generates personalized PDF certificates upon course completion and administers a badge system that awards visual milestones based on predefined criteria to enhance motivation.

- **Interactive Content Module:** Supports six activity types (flashcards, drag-and-drop, fill-in-the-blanks, memory games, timelines, H5P) with dedicated builder and viewer components, plus time-coded video questions and structured content blocks.
- **Communication and Notification Module:** Manages threaded lesson discussions, a contact form for inquiries, and a database-triggered notification system that alerts users on enrollment, progress, quiz attempts, lesson additions, and course publication.

1.5 Significance of Project

The development of the ACESS platform represents a significant step towards educational equity. By shifting the burden of accessibility from the user to the system architecture, the platform ensures that neurodivergent learners can focus entirely on comprehending the educational material rather than struggling with the interface.

For learners, the project provides a calm, predictable, and highly customizable environment that mitigates anxiety and cognitive fatigue. For educators, it offers a streamlined toolset that enforces accessibility best practices during content creation, alongside deep analytical insights into student engagement. Ultimately, ACESS demonstrates how modern web architecture can be leveraged to create a truly inclusive educational ecosystem applicable in both physical classrooms and remote learning scenarios.

1.6 Expected Output

The ACESS platform is expected to produce the following deliverables upon completion:

- **Adaptive Web Application:** A fully functional, production-ready learning

management system deployed on Vercel, accessible via standard web browsers and capable of dynamically adjusting its interface based on user accessibility profiles.

- **Role-Based Dashboards:** Three distinct interfaces for Learners, Educators, and Administrators, each with tailored navigation, permissions, and tooling appropriate to their respective responsibilities.
- **Accessibility Customization System:** A comprehensive set of user-configurable accessibility presets covering Dyslexia, ADHD, Autism, and Dyscalculia profiles, including typography control, theme switching, focus mode, text-to-speech, reduced motion, and simplified UI toggles.
- **Adaptive Recommendation Engine:** A rule-based engine that automatically analyzes quiz performance and generates personalized lesson recommendations to address identified knowledge gaps.
- **Course Authoring Tools:** A rich-text editor (Tiptap) integrated with media upload, interactive activity builders (flashcards, drag-and-drop, fill-in-the-blanks, memory games, timelines), and H5P content embedding for educators.
- **Quiz and Assessment System:** A complete assessment lifecycle including timed and untimed quizzes, automated grading, multiple attempt management, and answer review capabilities.
- **Analytics Dashboard:** Visual engagement metrics and progress tracking using Recharts, with per-student and per-course views for educators, plus platform-wide analytics for administrators.
- **Certificate and Achievement System:** Automated PDF certificate generation upon course completion and a badge-based achievement system to incentivize learner progress.
- **Notification System:** Database-triggered real-time notifications for enrollment, quiz results, lesson additions, and course publication across all user roles.

1.7 Conclusion

This chapter established the foundational context for the ACCESS project by identifying the critical accessibility barriers present in current educational platforms, defining the specific problems to be addressed, and outlining the objectives and scope of the proposed system. The significance of the project was discussed in terms of its potential impact on educational equity for neurodivergent learners, and the expected deliverables were enumerated. The next chapter presents a comprehensive literature review, examining existing theoretical frameworks, analyzing current Learning Management Systems, describing the project methodology, and detailing the software and hardware requirements necessary to realize the ACCESS platform.

CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY

2.1 Introduction

The literature review provides the academic and technical foundation upon which the ACES platform is conceptualized. This chapter synthesizes contemporary research regarding cognitive accessibility in digital education, evaluates established theoretical frameworks such as Universal Design for Learning (UDL), and rigorously analyzes existing Learning Management Systems (LMS). By comparing platforms like Moodle, Canvas, Blackboard, and Google Classroom against the specific requirements of neurodivergent learners, this chapter identifies the critical systemic gaps that justify the development of a natively adaptive, rule-based LMS architecture.

2.2 Theoretical Frameworks for Cognitive Accessibility

The design of digital interfaces for educational purposes must be grounded in empirically validated frameworks. The two primary pillars supporting the ACES architecture are the Universal Design for Learning (UDL) guidelines and the Web Content Accessibility Guidelines (WCAG) 2.2.

2.2.1 Universal Design for Learning (UDL)

Universal Design for Learning is a framework developed by CAST [2] aimed at optimizing teaching and learning for all individuals based on scientific insights into how humans learn. UDL is predicated on three primary principles:

- **Multiple Means of Engagement:** Recognizing that learners differ markedly in the ways they are motivated. For neurodivergent learners, particularly those with ADHD, motivation can be severely hampered by unstructured environments. ACCESS implements this by offering gamified achievements and structured, step-by-step progressive disclosure of course material to prevent cognitive overwhelm.
- **Multiple Means of Representation:** Learners vary in how they perceive and comprehend information. A Dyslexic learner may struggle with printed text but excel with audio. ACCESS implements this principle via integrated Text-to-Speech (TTS) capabilities and dynamic CSS injection that allows the user to radically alter the typography (e.g., using Atkinson Hyperlegible) and contrast ratios.
- **Multiple Means of Action and Expression:** Learners differ in how they navigate a learning environment and express what they know. ACCESS supports this by providing multiple quiz formats (scenario-based, multiple-choice) and allowing untimed assessments to reduce anxiety-induced performance degradation.

2.2.2 Web Content Accessibility Guidelines (WCAG) 2.2

While UDL provides the pedagogical framework, WCAG 2.2 [4] provides the strict, measurable technical parameters for digital accessibility. The ACCESS platform targets Level AA compliance. Key WCAG criteria directly addressed by ACCESS include:

- **Guideline 1.4.12 (Text Spacing):** The ability to override default text spacing (line height, letter spacing, word spacing) without loss of content or functionality. This is a critical requirement for Dyslexia, which ACCESS solves via its Tailwind CSS `@theme` variable injection.
- **Guideline 2.2.1 (Timing Adjustable):** Providing users the ability to turn off, adjust, or extend time limits. ACCESS allows educators to configure untimed quizzes specifically to accommodate learners requiring extended processing time.

- **Guideline 2.3.3 (Animation from Interactions):** Motion animation triggered by interaction can be disabled. The ACES system implements a `data-reduced-motion` attribute that globally disables all Framer Motion `<motion.div>` animations, critical for learners prone to vestibular disorders or severe distraction (ADHD).

2.3 Neurodivergent Profiles and Interface Interventions

Understanding the specific cognitive challenges faced by the target demographic is essential for architecting effective UI/UX interventions.

2.3.1 Attention Deficit Hyperactivity Disorder (ADHD)

ADHD is characterized by executive dysfunction, impacting working memory, task initiation, and sustained attention. In digital environments, learners with ADHD are highly susceptible to "saccadic fatigue" – the exhaustion caused by the eye constantly darting across a busy screen to parse disorganized information. **ACES Intervention:** The implementation of a strict "Focus Mode." This mode utilizes Next.js routing to conditionally hide the primary navigation sidebar, the global footer, and all extraneous dashboard widgets when a learner enters a `lesson_id` route. The content is restricted to a narrow, center-aligned column (maximum 65 characters per line), minimizing required eye movement and drastically reducing peripheral distraction.

2.3.2 Dyslexia

Dyslexia involves difficulties with accurate or fluent word recognition and poor spelling and decoding abilities. Research indicates that specific typographical adjustments can significantly improve reading velocity and comprehension [1]. **ACES Intervention:** The platform abandons reliance on browser-level zoom. Instead, it utilizes React Context to allow the user to select a "Dyslexia Profile." This injects CSS

variables that alter the global font family to specialized typefaces designed to prevent letter mirroring (e.g., confusing 'b' and 'd'), increases the base line-height to 1.75, and manipulates letter-spacing (tracking) to prevent visual crowding.

2.3.3 Autism Spectrum Disorder (ASD)

Learners on the Autism Spectrum often thrive in highly structured, predictable environments and can experience severe distress when confronted with unexpected auditory or visual stimuli. **ACCESS Intervention:** The platform enforces strict structural consistency. Educators utilizing the Tiptap rich-text editor are prevented from creating chaotic, ad-hoc layouts. Furthermore, the system defaults to suppressing autoplaying media and allows the user to globally toggle a "Reduced Motion" state, ensuring the interface remains a calm, predictable schedule of events.

2.4 Comparative Analysis of Existing LMS Platforms

To justify the development of ACCESS, an extensive evaluation of existing market-leading platforms was conducted. The platforms analyzed include Moodle, Canvas LMS, Blackboard Learn, and Google Classroom.

2.4.1 Moodle

Moodle is an open-source powerhouse, widely adopted globally [3]. Its greatest strength is its modularity. However, this is also its critical failing regarding cognitive accessibility. **Limitations:** Moodle's native interface is notoriously dense and text-heavy. While administrators can install accessibility plugins (like the 'Accessibility block'), these are bolt-on solutions. They require the learner to actively engage with a separate menu to adjust settings, adding cognitive load. Furthermore, Moodle relies on educators to manually identify struggling students and assign remedial work; it lacks a native, real-time heuristic engine to automate this process.

2.4.2 Canvas LMS

Canvas by Instructure represents a more modern, streamlined UI compared to Moodle. It utilizes a card-based dashboard that is visually appealing to neurotypical users. **Limitations:** Canvas achieves basic WCAG compliance but fails to provide deep cognitive accommodations. It lacks native Text-to-Speech (forcing reliance on OS-level tools) and does not offer granular control over typography (e.g., specific Dyslexia fonts or adjusted inter-letter spacing). The platform is highly linear; if a student fails a Canvas quiz, the system cannot automatically restructure the module path to enforce revision.

2.4.3 Blackboard Learn

Blackboard is an enterprise-grade solution focused heavily on synchronous learning integrations and comprehensive grading rubrics. **Limitations:** Blackboard's interface is often criticized for complex, deeply nested navigation menus (often three or four clicks deep to access a specific assignment file). For learners with executive dysfunction (ADHD), this navigational friction is a massive barrier to task initiation. Like Moodle, Blackboard's accessibility features are largely reactive (e.g., Blackboard ALLY checks documents for screen-reader compatibility) rather than proactive (dynamically altering the interface for the user).

2.4.4 Google Classroom

Google Classroom is a stripped-down, lightweight system focused on integration with the Google Workspace ecosystem (Docs, Drive, Forms). **Limitations:** While its minimalist design reduces initial visual clutter, it is too rudimentary to function as a fully adaptive LMS. It completely lacks advanced features such as rule-based learning paths, granular progress telemetry (tracking time-spent-per-lesson), or integrated cognitive profiles. It functions more as an assignment drop-box than a pedagogical support engine.

2.5 Comparison Matrix

Table 2.1 provides a structured summary comparing the features of the analyzed platforms against the proposed ACESS system.

Table 2.1: LMS Feature Comparison Matrix

Feature Domain	Moodle	Canvas	Blackboard	Google Classroom	Proposed (ACESS)
Native TTS	Plugin Req.	No	No	OS Reliant	Yes (Built-in)
Dynamic Fonts (Dyslexia)	Theme Req.	No	No	No	Yes (CSS Vars)
Automated Remedial Paths	No	No	No	No	Yes (Rule-based)
Focus Mode (ADHD)	No	No	No	Partial	Yes (Strict Route)
Gamification/Achievements	Plugin Req.	External App	No	No	Yes (Native)
WCAG 2.2 Level AA	Partial	Yes	Yes	Yes	Yes (Enforced)
Granular Analytics	Yes	Yes	Yes	No	Yes (Recharts)
Reduced Motion Toggle	No	No	No	No	Yes (Global State)

2.5.1 Alternative Approaches and Justification

Several alternative technical approaches were evaluated during the design of the ACESS platform before arriving at the selected architecture:

- **Monolithic Architecture vs. Decoupled Client-Server:** A monolithic Ruby on Rails or PHP Laravel application was considered for its simpler deployment model. However, this approach was rejected in favor of a decoupled Next.js frontend with Supabase backend-as-a-service, which allows the accessibility

engine to dynamically inject CSS variables on the client without full page reloads, preserving the fluid user experience critical for ADHD learners.

- **CSS-in-JS Libraries vs. Tailwind CSS:** Alternatives such as styled-components and Emotion were evaluated. While they offer dynamic styling, they introduce runtime overhead that degrades performance on low-end devices. Tailwind CSS v4 with Just-in-Time compilation and CSS custom properties was selected because it generates minimal production CSS and enables instantaneous theme switching through `data-attribute` selectors without JavaScript re-computation.
- **Document Database vs. Relational Database:** MongoDB was considered for its schema flexibility. However, the relational integrity enforced by PostgreSQL with foreign key constraints and Row-Level Security was deemed essential for ensuring data consistency across enrollments, progress tracking, and quiz attempts, particularly given the complex relational model required by the adaptive recommendation engine.
- **Traditional State Management vs. React Context:** Redux was considered for global state management but was deemed overly verbose for the accessibility settings use case. React Context with the `AccessibilityProvider` pattern was chosen for its simplicity, native React integration, and minimal boilerplate when propagating CSS theme attributes across the component tree.

2.6 Project Methodology

The ACES platform was developed using the **Agile Scrum** methodology, an iterative framework that emphasizes continuous delivery, regular stakeholder feedback, and adaptive planning. Scrum was selected over traditional sequential models such as Waterfall because designing for cognitive accessibility is an inherently empirical process – assumptions about the effectiveness of specific interface interventions (e.g., a Dyslexia font or Focus Mode layout) must be validated through iterative user testing rather than deferred to the final stages of development.

The development lifecycle was organized into six sprints, each spanning two weeks:

- **Sprint 0 – Requirements and Scaffolding:** Definition of the product backlog based on WCAG 2.2 guidelines and UDL principles. Establishment of the Next.js project structure, Supabase project initialization, and PostgreSQL schema design.
- **Sprints 1-2 – Core Architecture:** Implementation of user authentication with Supabase Auth, database schema normalization with Row-Level Security policies, and basic role-based routing via middleware.
- **Sprints 3-4 – Feature Development:** Development of the Educator authoring interface (Tiptap editor, course creation, quiz builder) and the Learner experience (course enrollment, lesson viewing, progress tracking).
- **Sprints 5-6 – Advanced Features and Refinement:** Integration of the rule-based adaptive recommendation engine, accessibility personalization engine with CSS variable injection, analytics dashboards, certificate generation, and notification triggers.

Each sprint concluded with a review and retrospective, ensuring continuous alignment with project objectives. A detailed breakdown of the Agile implementation is provided in Chapter 3.

2.7 Project Requirements

2.7.1 Software Requirements

The following software tools and technologies were utilized in the development and deployment of the ACCESS platform:

- **Visual Studio Code:** Primary integrated development environment for writing and debugging code, with extensions for TypeScript, Tailwind CSS, and ESLint integration.
- **Node.js:** JavaScript runtime environment required to execute the Next.js development server and build toolchain.

- **Git and GitHub:** Version control system for source code management and collaboration, with the repository hosted on GitHub for continuous integration with Vercel.
- **Supabase CLI:** Command-line tool for local database migrations, schema management, and TypeScript type generation for the PostgreSQL backend.
- **Vercel:** Cloud platform for deploying the Next.js application, providing serverless edge functions, automatic HTTPS, and continuous deployment from the GitHub repository.
- **Next.js 16:** React framework with App Router providing server-side rendering, static generation, and middleware-based route protection.
- **TypeScript:** Statically typed superset of JavaScript used throughout the codebase for improved code quality and maintainability.
- **Tailwind CSS v4:** Utility-first CSS framework with Just-in-Time compilation, enabling dynamic theme injection via CSS custom properties for accessibility customization.
- **Supabase (PostgreSQL):** Backend-as-a-service providing a managed PostgreSQL database with Row-Level Security, authentication, and file storage capabilities.
- **Framer Motion:** Animation library for React used to implement smooth transitions while respecting the user's reduced-motion accessibility preference.
- **Tiptap:** Headless rich-text editor built on ProseMirror, integrated into the Educator authoring workflow for creating structured lesson content.
- **Recharts:** Charting library for React used to render engagement analytics and progress visualizations on educator and administrator dashboards.
- **Figma:** UI/UX design tool used to prototype interface layouts and accessibility component mockups prior to implementation.

Placeholder - Insert Image Here

Figure 2.1: Software tools used in ACCESS development

2.7.2 Hardware Requirements

The following hardware specifications were used during development and are recommended for optimal deployment:

- **Development Workstation:** A personal computer with a minimum of 8 GB RAM, a multi-core processor (Intel Core i5 or equivalent), and a solid-state drive with at least 256 GB of storage for running the development server, database migrations, and local testing.
- **Display:** A monitor with a minimum resolution of 1920x1080 pixels to adequately preview responsive layouts and accessibility adjustments across viewport sizes.
- **Network Connectivity:** A stable broadband internet connection with at least 10 Mbps download speed for accessing cloud services (Supabase, Vercel, GitHub) and conducting user acceptance testing.
- **Server Infrastructure:** Cloud-based infrastructure provided by Vercel (edge

functions and static hosting) and Supabase (managed PostgreSQL and file storage), eliminating the need for dedicated physical server hardware.

2.7.3 Other Requirements

- **Supabase Account:** A cloud-hosted Supabase project for database management, authentication configuration, and storage bucket policies.
- **GitHub Account:** Repository hosting for source code version control and integration with Vercel for continuous deployment.
- **Vercel Account:** Deployment platform account linked to the GitHub repository for automated builds and hosting.
- **SMTP Email Service:** A Gmail SMTP configuration for sending password reset emails and account verification notifications via Nodemailer.
- **Web Browser:** A modern browser (Google Chrome, Mozilla Firefox, or Microsoft Edge) for testing and accessing the deployed application.

2.8 Project Schedule and Milestones

The ACES project was executed across six Agile Scrum sprints, each spanning two weeks, for a total development duration of twelve weeks. Table 2.2 summarizes the milestone schedule.

Table 2.2: Project Schedule and Milestones

Sprint	Focus Area	Key Deliverables
0	Requirements and Scaffolding	Product backlog, Next.js project setup, Supabase initialization, database schema design
1	Core Authentication	User registration and login, role-based middleware, email verification flow
2	Database and Security	Row-Level Security policies, storage bucket configuration, data dictionary implementation
3	Educator Authoring	Course creation interface, Tiptap editor integration, quiz builder, media upload
4	Learner Experience	Course enrollment, lesson viewer, progress tracking, Focus Mode implementation
5	Adaptive and Analytics	Recommendation engine, accessibility presets, analytics dashboards, certificates, notifications

The Gantt chart in Figure 2.2 provides a visual representation of the project timeline, illustrating the scheduling and duration of each sprint.

Placeholder - Gantt Chart Here

Figure 2.2: Gantt chart of ACESS development sprint schedule

2.9 Summary

The literature review demonstrates that while current LMS platforms provide robust administrative and content-hosting capabilities, they systematically fail to address the specific cognitive requirements of neurodivergent learners. Reliance on external plugins, static content delivery models, and dense navigation hierarchies induce significant extraneous cognitive load. The theoretical frameworks of UDL and WCAG 2.2 demand a more integrated approach. The comparative analysis definitively justifies the engineering of the ACCESS platform: a system built from the database schema upward to be natively adaptive, utilizing dynamic CSS injection and rule-based server logic to provide an equitable, highly personalized learning environment. The project methodology, requirements, and schedule outlined in this chapter establish the roadmap for implementing the ACCESS system in the subsequent phases.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Introduction

This chapter transitions the theoretical justifications established in the literature review into concrete software engineering specifications. It begins with a detailed problem analysis of existing educational platforms, followed by a comprehensive requirements analysis covering data, functional, non-functional, and other requirements. The chapter then presents the system architecture, module design, and deployment considerations necessary to execute a secure, highly accessible Learning Management System.

3.2 Problem Analysis

The ACESS platform is designed to address specific, systemic shortcomings in current educational technology. While the literature review in Chapter 2 established the general limitations of platforms such as Moodle, Canvas, Blackboard, and Google Classroom, this section analyzes the concrete operational problems that directly influenced the architectural decisions of ACESS.

3.2.1 Current System Scenario

Existing Learning Management Systems typically operate under a one-size-fits-all paradigm. Learners are presented with a fixed interface regardless of their cognitive profile, forcing neurodivergent users to seek third-party accommodations such as

browser extensions for text-to-speech, external tools for font customization, or manual workarounds for managing visual clutter. This places the burden of accessibility on the learner rather than the system.

From a data flow perspective, current platforms process learner interactions in a linear fashion: enrollment, content delivery, assessment, and completion. They lack the feedback loops necessary to detect when a learner is struggling and respond adaptively. For example, if a student performs poorly on a quiz about database normalization, the system continues to present the next topic as though no intervention is needed.

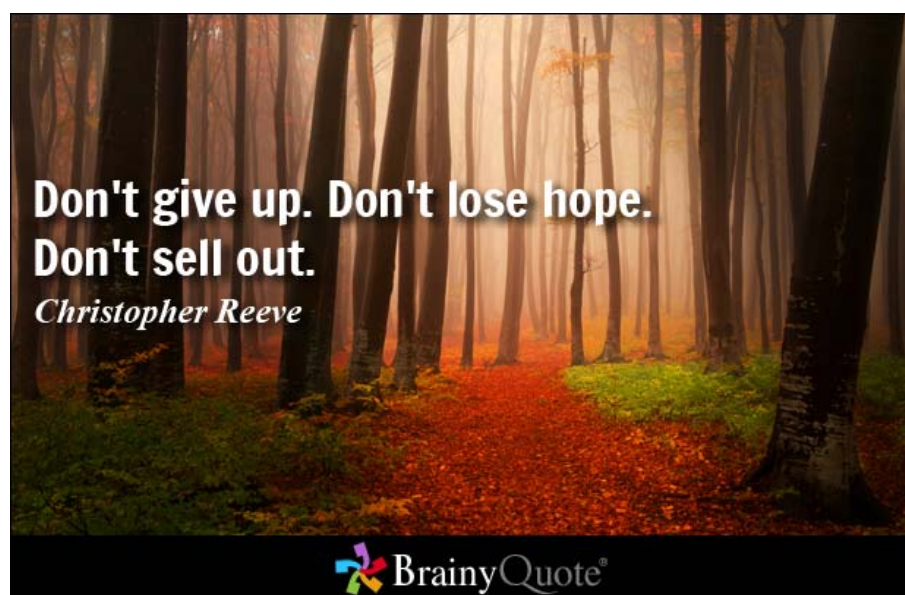


Figure 3.1: Placeholder for Current System Problem Analysis Diagram

3.2.2 Detailed Problem Breakdown

The problems identified in Chapter 1 are analyzed here in greater technical depth:

1. **Rigid Typography and Layout:** Standard LMS platforms use fixed CSS stylesheets that do not respond to individual user needs. Learners with Dyslexia require specific typographic conditions – increased letter spacing (tracking) of at least 0.12em, line height of 1.5 to 1.8, and sans-serif fonts such as Atkinson Hyperlegible designed to prevent letter confusion (e.g., differentiating 'b' from

'd'). Without these adjustments, reading velocity decreases and cognitive fatigue accelerates, leading to early lesson abandonment.

2. **Cognitive Overload from Navigation:** Platforms like Moodle and Blackboard present deeply nested navigation menus (often three to four levels deep) combined with dense dashboard widgets showing calendars, announcements, grades, and messages simultaneously. For learners with ADHD, this multi-stimulus environment triggers rapid attention fragmentation, making it difficult to sustain focus on a single learning objective. The absence of a distraction-free mode means learners must manually close or minimize interface elements.
3. **Lack of Adaptive Remediation:** Current assessment systems grade quizzes and record scores but do not analyze performance patterns to generate corrective action. When a learner fails a prerequisite concept, the system does not automatically unlock remedial content or recommend revision. This linear progression model assumes uniform learner readiness, which is particularly detrimental for neurodivergent students who may require repeated exposure to foundational topics before advancing.

3.3 Requirement Analysis

3.3.1 Data Requirement

The ACES platform requires a robust relational data model to manage the complex relationships between users, courses, lessons, assessments, and accessibility preferences. The system must store and process the following categories of data:

- **User Data:** Authentication credentials (email, password hash), profile information (name, avatar, role), and notification preferences. User data must be isolated by role with strict access controls.
- **Accessibility Data:** Per-user cognitive profile settings including disability type, preferred font family and size, theme (light, dark, high-contrast, soft), line spacing,

text-to-speech and caption toggles, reduced motion preference, simplified UI mode, and keyboard navigation flags. These settings must persist across sessions and apply globally to all interface components.

- **Curriculum Data:** Hierarchical course structures (courses, chapters, lessons) with rich-text HTML content, video assets, transcripts, and metadata such as difficulty level, estimated duration, and prerequisite relationships that drive the adaptive engine.
- **Assessment Data:** Quiz configurations (time limits, pass thresholds, max attempts), question banks with multiple-choice and scenario formats, answer options with correctness flags, and learner attempt records including scores and answer selections.
- **Telemetry Data:** Granular engagement metrics including lesson view status, time spent per lesson (in seconds), enrollment status (active, completed, dropped), and quiz attempt history with pass/fail outcomes.
- **Content and Achievement Data:** Interactive activity configurations (flashcards, drag-and-drop, memory games, timelines), H5P embedded content, certificate records with unique reference codes, and badge/achievement definitions and user progress toward them.
- **Notification and Communication Data:** System-generated notifications triggered by enrollment, quiz attempts, lesson additions, and course publications, along with lesson-level threaded comments and contact form submissions.

The detailed database schema and data dictionary for these entities are presented in Chapter 4, Section 4.3.

3.3.2 Functional Requirement

Functional requirements define the explicit behaviors, processes, and services the system must execute. They are derived directly from the system's operational architecture and are categorized by user role.

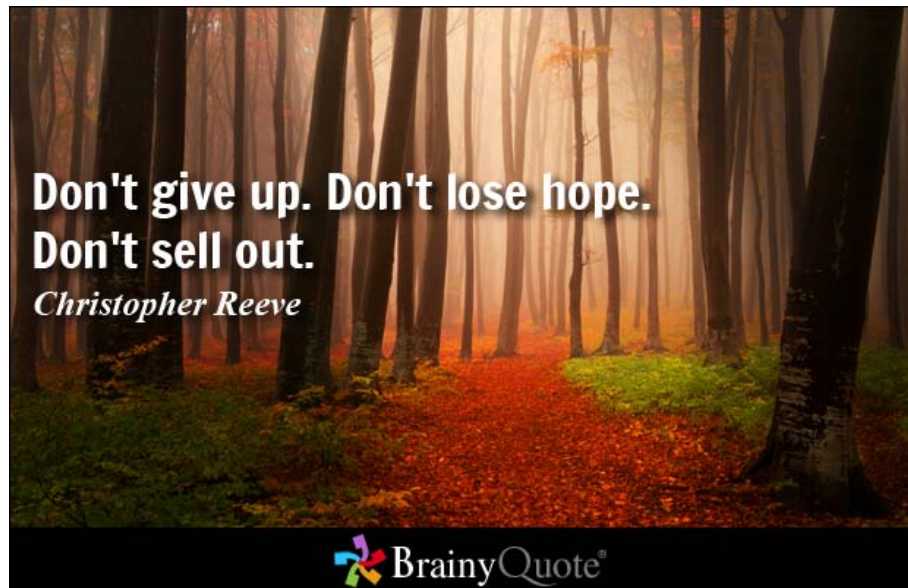


Figure 3.2: Placeholder for Use Case Diagram

Use Case Diagram Figure 3.2 visually maps the interactions between the primary actors (Student, Educator, Admin) and the system's functional boundaries, which are explicitly detailed in the subsequent tables.

Table 3.1: Functional Requirements: Student Module

Req ID	Description	Actor	Inputs	Outputs
FR-STU-01	Registration and secure login.	Student	Email, Password	Authenticated JWT Session.
FR-STU-02	Accessibility profile selection.	Student	Theme, Font, TTS preferences	Updated <code>user_accessibility_settings</code> record; UI dynamically alters.
FR-STU-03	Course discovery and enrollment.	Student	<code>course_id</code> selection	INSERT into <code>enrollments</code> ; unlocks course content.
FR-STU-04	Lesson viewing with Focus Mode.	Student	Navigation to <code>/lesson/[id]</code>	Renders sanitized HTML; hides global navigation elements.
FR-STU-05	Interactive video questions.	Student	Video timestamp trigger	Pauses video, renders embedded question overlay.
FR-STU-06	Quiz participation.	Student	Selected <code>quiz_options</code>	Evaluates score; INSERTs into <code>quiz_attempts</code> .
FR-STU-07	Automated Progress tracking.	Student	Dwell time on lesson route	UPDATEs <code>time_spent</code> in <code>lesson_progress</code> .
FR-STU-08	Achievement earning.	Student	100% module completion	Unlocks gamified badge in UI.
FR-STU-09	Certificate generation/viewing.	Student	100% course completion	Generates verifiable PDF; displays download link.

Student Functions

Table 3.2: Functional Requirements: Educator Module

Req ID	Description	Actor	Inputs	Outputs
FR-EDU-01	Course creation and editing.	Educator	Title, metadata, difficulty_level	INSERT/UPDATE courses table.
FR-EDU-02	Lesson creation via Tiptap.	Educator	Rich-text content	Sanitized HTML string saved to lessons.content_html.
FR-EDU-03	Quiz and Activity creation.	Educator	Question text, Correct option flag	Populates quiz_questions and quiz_options linked via FK.
FR-EDU-04	Asset management.	Educator	MP4/Image files	Uploads to Supabase Storage; returns secure URL.
FR-EDU-05	Student monitoring & Analytics.	Educator	Selection of course_id	Renders Recharts graphs aggregating lesson_progress data.
FR-EDU-06	Achievement configuration.	Educator	Threshold values	Maps achievements to specific course_milestones.

Educator Functions

Table 3.3: Functional Requirements: Admin Module

Req ID	Description	Actor	Inputs	Outputs
FR-ADM-01	User role management.	Admin	Target user_id, new role string	UPDATEs users.role allowing bypass of middleware RBAC.
FR-ADM-02	Course moderation pipeline.	Admin	Review action (Approve/Reject)	UPDATEs courses.status from pending_review to published.
FR-ADM-03	Accessibility preset management.	Admin	Global CSS variable thresholds	Updates system default fallbacks for Dyslexia/ADHD themes.
FR-ADM-04	System course creation.	Admin	Course metadata	Flags course as system_course=true, bypassing educator limits.
FR-ADM-05	Global Analytics monitoring.	Admin	Date range filters	Aggregates platform-wide engagement metrics.

Admin Functions

3.3.3 Non-Functional Requirement

Non-functional requirements (NFRs) specify the critical quality attributes and systemic constraints that govern the operation of the ACES platform.

Performance

- **Page Response Time:** Next.js Client-side route transitions must execute within **300ms** to prevent cognitive disruption and maintain user focus.
- **Course Loading:** Server-side data fetching for initial course loads must resolve within **800ms**, leveraging edge caching where applicable.
- **Video Loading:** Embedded instructional videos must commence playback within **2 seconds** utilizing adaptive bitrate streaming via Supabase Storage CDNs.
- **Concurrent Users:** The PostgreSQL database, utilizing connection pooling (PgBouncer), must support a minimum of **500 concurrent** intensive operations (e.g., simultaneous quiz submissions) without degrading frontend TTFB (Time to First Byte).

Security

- **Authentication:** All user sessions must be cryptographically secured using JSON Web Tokens (JWT) managed by Supabase Auth, with a mandatory 15-minute inactivity timeout.
- **Authorization (RBAC):** The Next.js 'proxy.ts' middleware must definitively intercept and route requests based on the 'role' claim within the JWT, achieving zero-trust client routing.

- **Database Protection:** Strict Row-Level Security (RLS) must be enabled on 100% of tables containing Personally Identifiable Information (PII) or cognitive telemetry, ensuring isolation at the storage layer.
- **Input Validation:** All API endpoints must utilize Zod schema validation to sanitize incoming payloads, preventing SQL injection and Cross-Site Scripting (XSS).

Accessibility

- **WCAG Compliance:** The platform must strictly adhere to **WCAG 2.2 Level AA** standards. Minimum contrast ratios must exceed 4.5:1 for standard text.
- **Dyslexia Support:** The system must natively provide the Atkinson Hyperlegible typeface and allow users to dynamically increase line-spacing to 1.75em.
- **ADHD & Autism Support:** The implementation of a 'data-reduced-motion' global attribute must programmatically disable all CSS and Framer Motion animations. "Focus Mode" must isolate content to a 65-character maximum width to reduce saccadic fatigue.
- **Screen Reader Compatibility:** All custom UI components (built via shadcn/ui) must natively handle WAI-ARIA states (e.g., 'aria-expanded', 'aria-hidden') and support complete keyboard navigation.

Usability

- **Consistent Navigation:** The primary navigational architecture must remain structurally identical across all modules to ensure predictability for Autistic learners.
- **Error Prevention:** Destructive actions (e.g., an Educator deleting a course) must require a confirmed multi-step verification modal to prevent accidental data loss.

Reliability

- **Data Integrity:** The database schema must strictly enforce foreign key constraints (ON DELETE CASCADE where appropriate) to prevent orphaned records (e.g., a `lesson_progress` record existing without a valid `enrollment_id`).
- **Backup Strategy:** The Supabase infrastructure must be configured to execute automated, daily Point-in-Time Recovery (PITR) backups of the entire PostgreSQL volume.

Maintainability and Scalability

- **Modular Architecture:** The codebase must adhere to strict separation of concerns, decoupling the data-fetching logic (Server Components) from the interactive accessibility logic (Client Components).
- **Database Normalization:** The schema must adhere to the Third Normal Form (3NF) to eliminate data redundancy, ensuring the database scales efficiently as course and user volume increases.

3.3.4 Others Requirement

The software and hardware tools selected for the ACES platform, as listed in Chapter 2, Section 2.4, are justified by the following architectural considerations:

- **Next.js 16 and React:** Server-side rendering and React Server Components reduce initial JavaScript payload, critical for rapid page loads that prevent cognitive drop-off for ADHD learners. The App Router provides middleware-based route protection that enforces role-based access at the edge.
- **Supabase with PostgreSQL:** Row-Level Security provides data isolation at the database level rather than relying solely on application-layer guards, ensuring that cognitive telemetry and assessment data remain cryptographically isolated per user.

- **Tailwind CSS v4:** Just-in-Time compilation enables dynamic CSS variable injection via `data-attribute` selectors, allowing instantaneous theme and typography switching without JavaScript re-evaluation – essential for real-time accessibility adjustments.
- **TypeScript:** Static typing across the full stack reduces runtime errors in critical paths such as quiz scoring and accessibility profile application, where a type mismatch could result in incorrect interface rendering.
- **Vercel:** Serverless edge deployment provides global low-latency access to the platform, which is particularly important for learners whose cognitive focus may be disrupted by slow page transitions.

3.4 System Architecture

The ACES platform employs a modern, decoupled client-server architecture, leveraging edge computing paradigms to maximize performance and security.

3.4.1 Context Diagram

The Context Diagram establishes the highest-level view of the system, illustrating the boundaries of the ACES platform and its interactions with external entities.

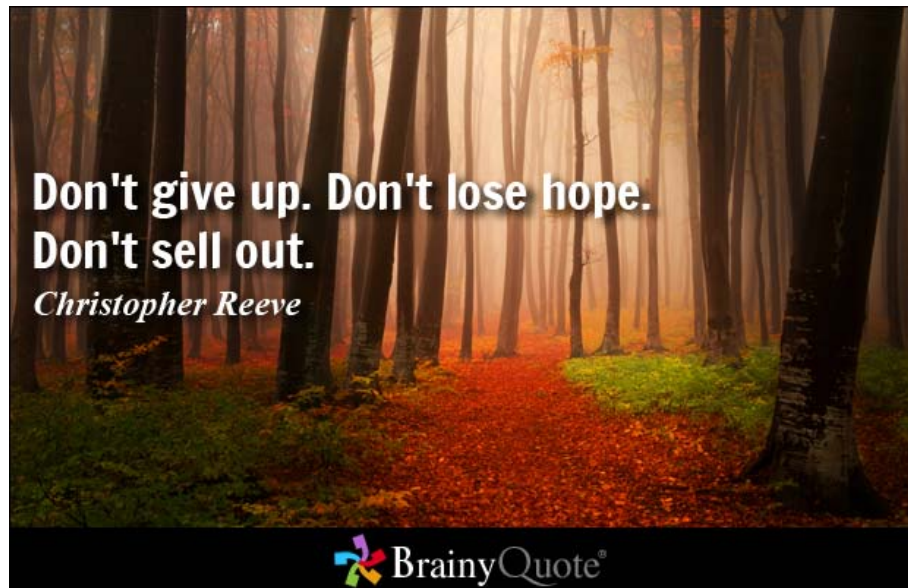


Figure 3.3: Placeholder for Context Diagram

As depicted in Figure 3.3, the ACESS system acts as the central processing hub. The primary external entities are the Learner (who consumes content and provides telemetry), the Educator (who authors content and configures assessments), and the Administrator (who governs system integrity). The system also interfaces with external secure email providers (for authentication/password resets) and potentially external text-to-speech APIs, though native browser synthesis is prioritized.

3.4.2 System Architecture Diagram

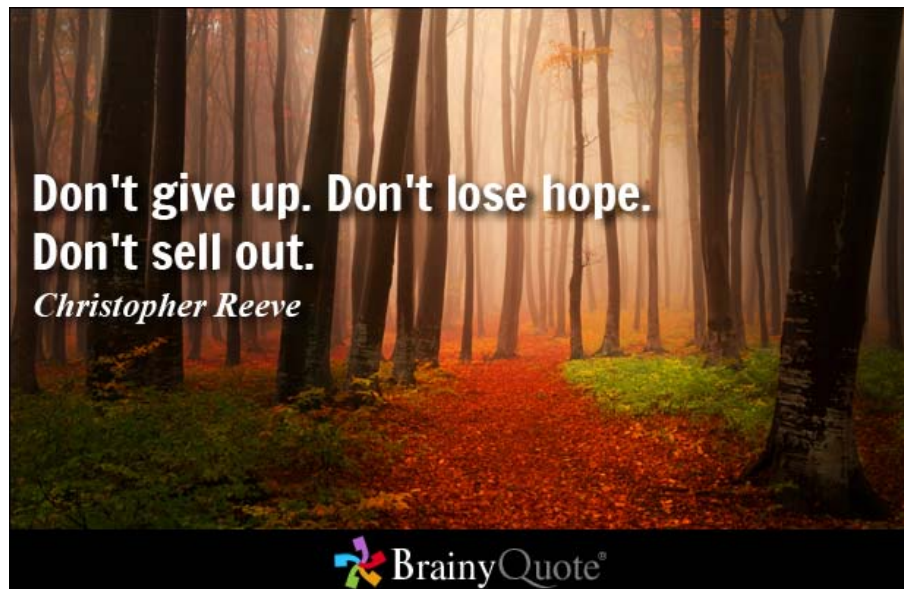


Figure 3.4: Placeholder for System Architecture Diagram

Figure 3.4 details the three-tier decoupled architecture:

- **Frontend Architecture (Next.js 16 & React):** The presentation layer is built on the Next.js App Router. It utilizes React Server Components (RSC) to execute data fetching securely on the server, drastically reducing the JavaScript payload delivered to the client. This is crucial for rapid page loads, preventing cognitive drop-off for ADHD learners. Client components (“use client”) are restricted to highly interactive elements, such as the Tiptap editor and interactive quiz interfaces. The UI is styled utilizing Tailwind CSS v4, which enables the dynamic injection of ‘@theme’ variables for accessibility toggling.
- **Middleware & Authentication Layer:** A Next.js Edge Middleware intercepts every incoming request. It interfaces with ‘@supabase/ssr’ to validate JWT cookies, extracting the user’s ‘role’. The middleware enforces strict Role-Based Access Control (RBAC), instantly redirecting unauthorized attempts (e.g., a Learner attempting to access ‘/educator/dashboard’) before the React tree is even evaluated.
- **Backend & Database Architecture (Supabase):** Supabase acts as the

comprehensive Backend-as-a-Service (BaaS). It provides a managed PostgreSQL database, handling complex relational queries. Crucially, data security is not solely reliant on the API layer; PostgreSQL Row-Level Security (RLS) policies are inscribed directly into the database schema, ensuring users can only ‘SELECT’, ‘INSERT’, or ‘UPDATE’ records they cryptographically own.

- **Storage Architecture:** Supabase Storage buckets manage unstructured assets (course thumbnails, PDF certificates, embedded videos). These buckets are governed by parallel RLS policies, ensuring private assets remain inaccessible to unauthenticated requests.

3.4.3 Deployment Diagram

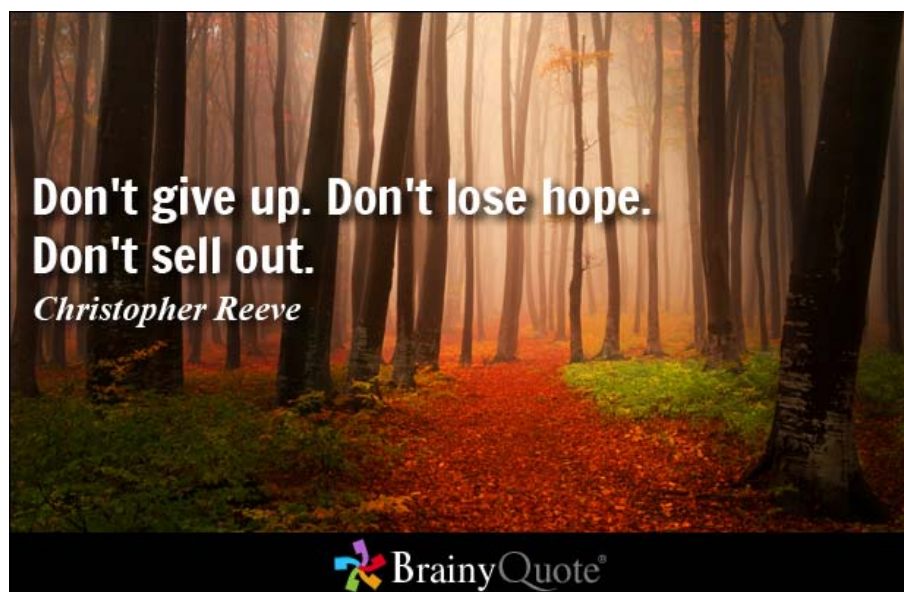


Figure 3.5: Placeholder for Deployment Diagram

The Deployment Diagram (Figure 3.5) illustrates the physical distribution of the system components. The Next.js frontend is deployed to a serverless edge network (e.g., Vercel or AWS Amplify), ensuring high availability and low latency globally. The Supabase backend operates on dedicated cloud infrastructure, exposing a secure REST/GraphQL API via PostgREST, which the Next.js server components consume securely.

3.5 Module Design

The system is logically partitioned into discrete, highly cohesive modules to facilitate maintainability.

- **Student Module:** Encapsulates all logic related to course discovery, enrollment (`enrollments` table), lesson consumption (`lesson_progress`), and quiz execution (`quiz_attempts`).
- **Educator Module:** Manages the creation pipeline. Handles the Tiptap editor state, quiz generation (`quiz_questions`, `quiz_options`), and the uploading of media assets to Supabase Storage.
- **Admin Module:** Provides a high-level oversight interface for user management, role reassignment, and global accessibility preset configurations.
- **Accessibility Module:** A globally available React Context Provider (`AccessibilityProvider`). It subscribes to the user's preferences in the `user_accessibility_settings` table and dynamically manipulates the DOM root attributes (e.g., `data-theme="dyslexia"`, `data-reduced-motion="true"`).
- **Interactive Activity Module:** Governs the execution of varied assessment types, including standard multiple-choice quizzes and embedded video questions, immediately evaluating responses against cryptographic hashes in the backend.
- **Analytics Module:** Aggregates data from `lesson_progress` and `quiz_attempts` to render visualizations via the Recharts library, providing educators with actionable intelligence on cohort performance.

3.6 Summary

This chapter analyzed the engineering specifications required to build the ACES platform. It began with a detailed problem analysis of existing educational systems,

followed by a comprehensive requirements analysis covering data, functional, non-functional, and other requirements. The system architecture was presented through Context, Architecture, and Deployment diagrams, and the modular decomposition of the system was described. The subsequent chapter will translate these requirements into concrete database schemas and interface designs.

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

This chapter translates the conceptual requirements established in the previous phase into concrete, highly granular engineering designs. It is structured according to the High-Level Design and Detailed Design phases. High-Level Design encompasses behavioral and interaction diagrams, user interface design specifications, and the conceptual database model. Detailed Design covers the software component architecture, physical database implementation, and systemic implementation details of the ACCESS platform.

4.2 High-Level Design

4.2.1 Behavioral Design (Activity Diagrams)

Activity diagrams model the workflow of the system, illustrating the procedural logic executed by the different actors.

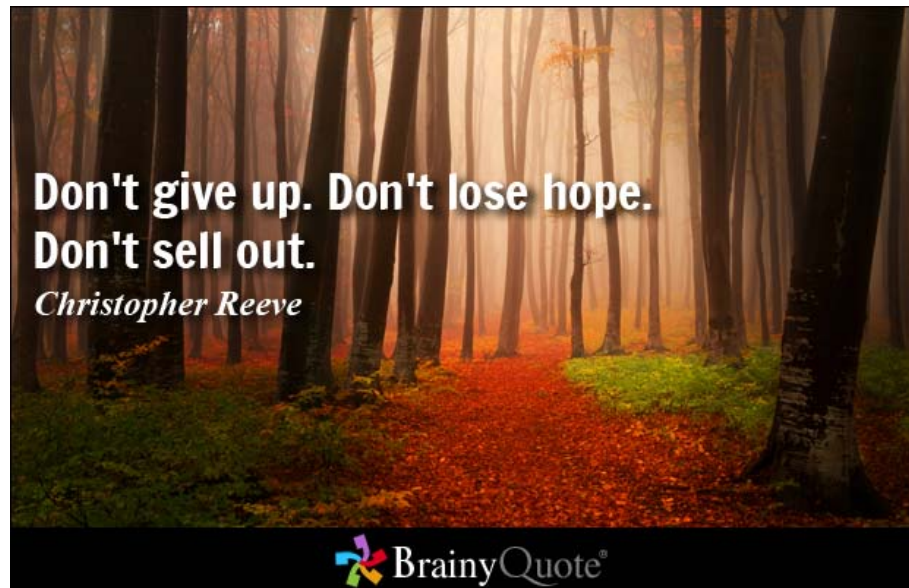


Figure 4.1: Placeholder for Student Activity Diagram

Student Activity Diagram As shown in Figure 4.1, the Student workflow is highly structured. Upon authentication, the system evaluates the `user_accessibility_settings`. If incomplete, the user is routed to an onboarding profile setup. Once the dashboard loads, the core loop involves browsing the catalog, initializing an enrollment, and sequentially consuming lessons. The critical branching logic occurs post-assessment: if a quiz score is below the predefined threshold, the system diverges from the linear path, querying the `recommendations` table and forcing the student to review foundational material before progressing.

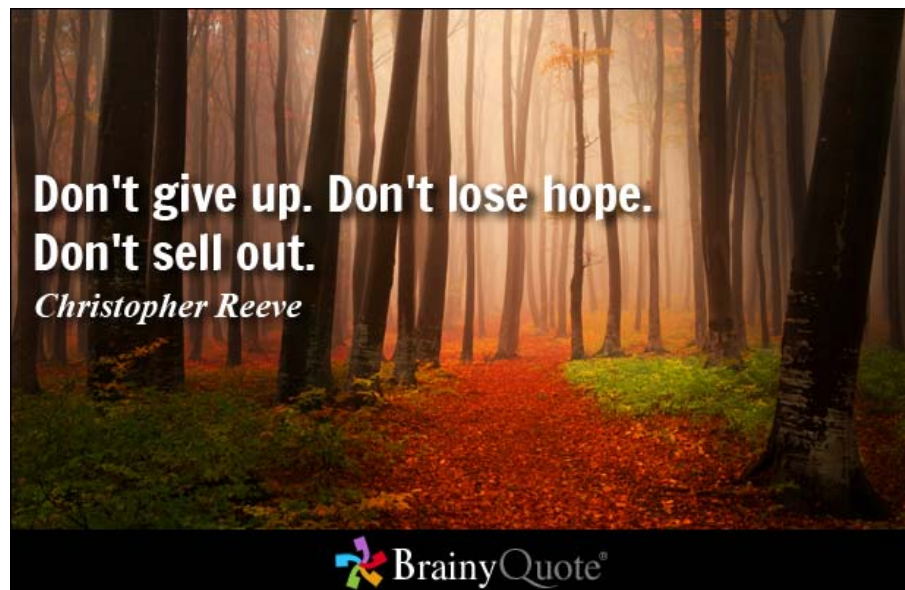


Figure 4.2: Placeholder for Educator Activity Diagram

Educator Activity Diagram Figure 4.2 maps the Educator’s authoring workflow. The process begins with establishing course metadata and structure (`course_chapters`). The educator then enters a loop of content creation: drafting lessons via Tiptap, uploading video assets, and defining `quiz_questions`. A critical validation node exists before publishing: the system checks for mandatory accessibility flags (e.g., verifying video transcripts are attached). Upon passing validation, the course status updates to `pending_review`.

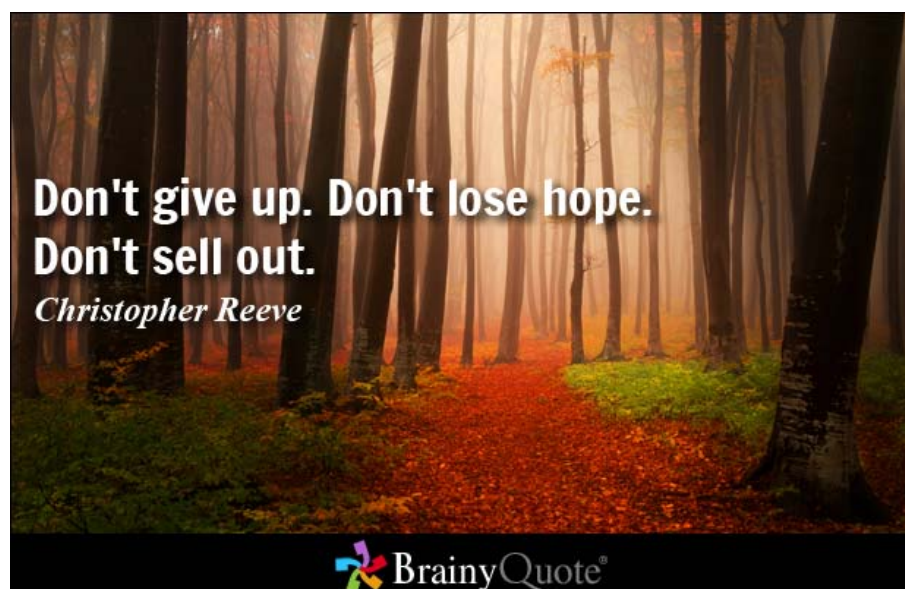


Figure 4.3: Placeholder for Administrator Activity Diagram

Administrator Activity Diagram The Administrator workflow (Figure 4.3) is primarily reactive. The admin monitors the moderation queue. When a course enters `pending_review`, the admin evaluates it against overarching accessibility guidelines. If rejected, a notification payload is dispatched to the educator. If approved, the course is published. Furthermore, the admin workflow includes auditing user logs and modifying global system variables.

4.2.2 Interaction Design (Sequence Diagrams)

Sequence diagrams detail how objects interact over time to execute specific Use Cases, mapping the exact API calls between the Next.js client, the Edge Middleware, and the Supabase backend.

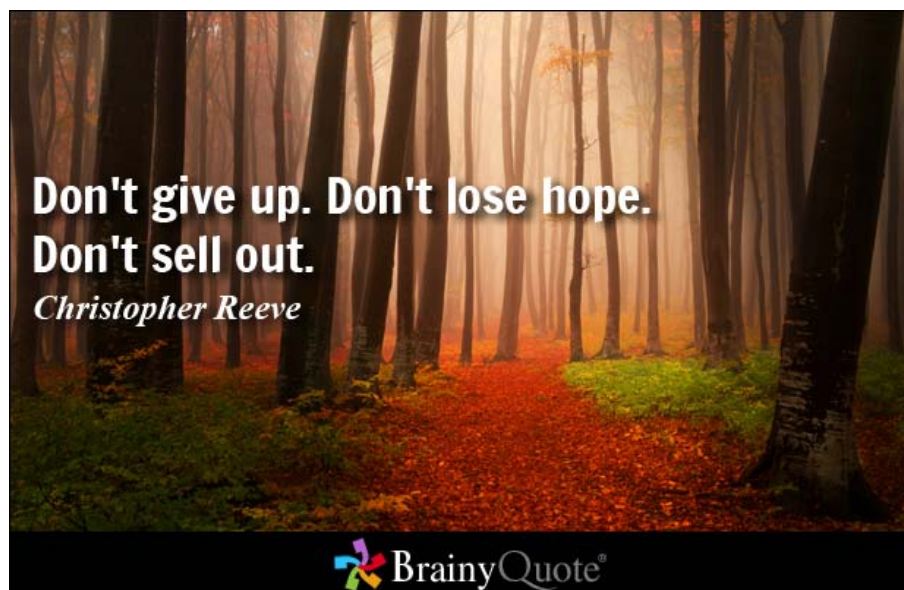


Figure 4.4: Placeholder for Course Enrollment Sequence Diagram

Sequence: Course Enrollment Figure 4.4 illustrates the enrollment transaction. 1. The Learner Client sends a `POST /api/enroll` request containing the `course_id`. 2. The Edge Middleware intercepts, verifying the JWT session. 3. The API queries Supabase to check for existing enrollments (preventing duplicates). 4. An `'INSERT'` operation is executed on the `'enrollments'` table. 5. A PostgreSQL Trigger fires, executing

a webhook that inserts a record into the 'notifications' table, alerting the educator. 6. The client is redirected to the '/course/[id]/learn' route.

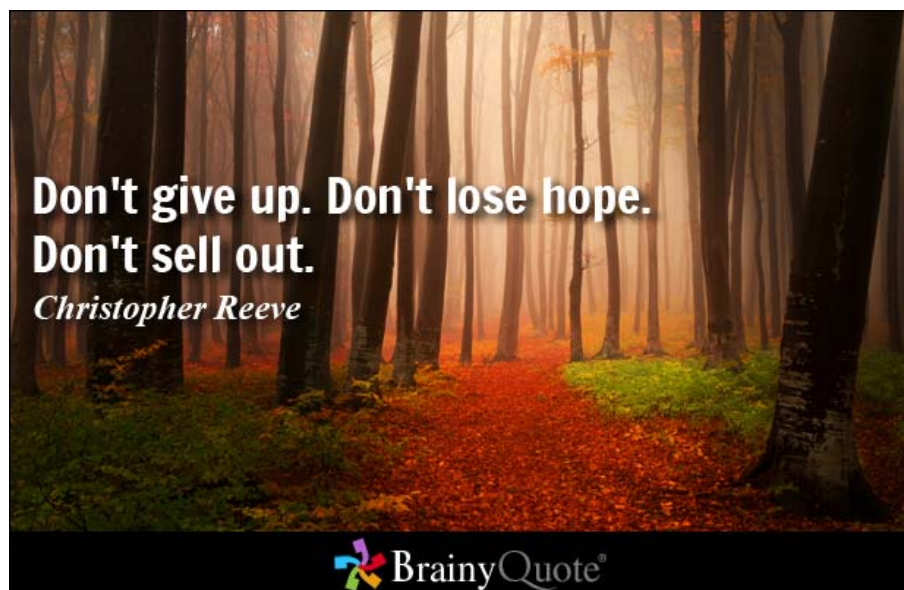


Figure 4.5: Placeholder for Lesson Learning Sequence Diagram

Sequence: Lesson Learning & Telemetry Figure 4.5 maps the continuous telemetry tracking. When a learner mounts a lesson component, an `INSERT` establishes a `lesson_progress` record. As the user dwells on the page, a client-side `setInterval` function fires every 30 seconds, dispatching an asynchronous `PATCH` request to update the `time_spent_learning` integer in the database. This guarantees highly accurate engagement metrics even if the user closes the browser abruptly.

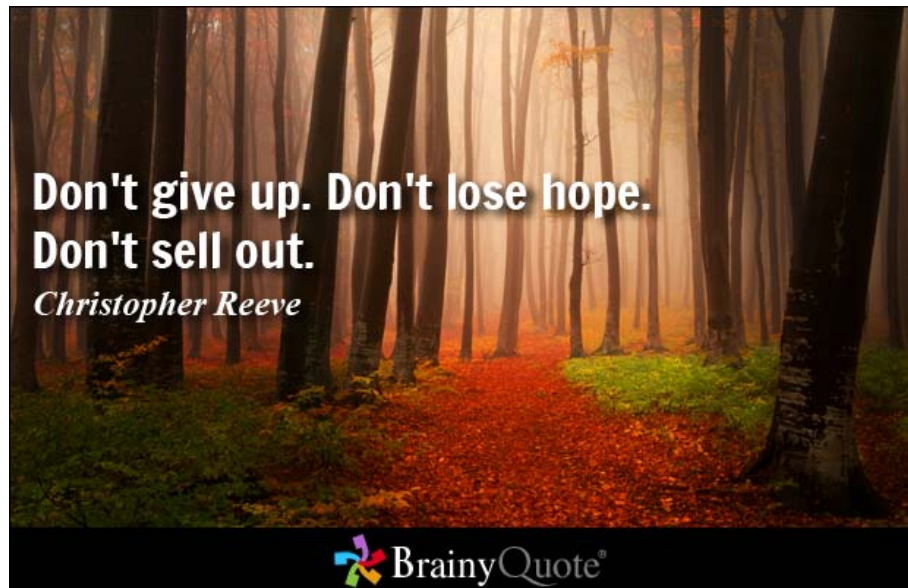


Figure 4.6: Placeholder for Quiz Attempt Sequence Diagram

Sequence: Quiz Attempt & Adaptive Recommendation Figure 4.6 details the most complex systemic interaction: the adaptive engine. 1. The Learner submits a JSON payload of `selected_option_id` values. 2. The backend securely evaluates these against the `is_correct` flags in `quiz_options`, ensuring the client never possesses the answer key. 3. The aggregate `score_pct` is inserted into `quiz_attempts`. 4. The server evaluates the score against `quizzes.pass_threshold_pct`. 5. If failed, the backend queries `lessons.prerequisite_lesson_id`. 6. An `INSERT` is executed on the `recommendations` table, flagging the prerequisite for mandatory review.

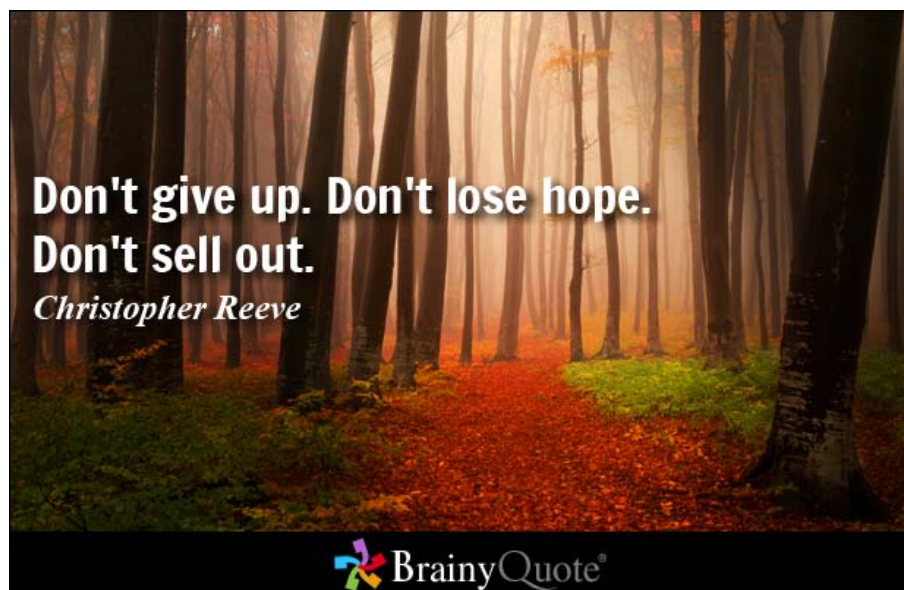


Figure 4.7: Placeholder for Course Creation Sequence Diagram

Sequence: Course Creation & Media Upload Figure 4.7 shows the Educator authoring pipeline. When an educator uploads a video, the Next.js API requests a signed upload URL from Supabase Storage. The client then uploads the binary blob directly to the `course-assets` bucket, bypassing the Next.js server to prevent memory bottlenecks. The resulting public URL is then stored within the `lessons.video_url` attribute.

4.2.3 User Interface Design

The interface architecture of ACESS is organized into three role-specific shells, each with distinct navigation structures, input mechanisms, and output presentations.

Navigation Design The navigation architecture follows a role-based shell pattern enforced by Next.js middleware:

- **Public Navigation:** Unauthenticated users access the landing page, login, signup, and password reset routes. The landing page features a hero section, feature highlights, system capabilities showcase, and a role-based demo modal.

- **Learner Navigation:** The learner shell (`LearnerShell`) contains a persistent collapsible sidebar with links to Dashboard, Courses, Progress, Certificates, Achievements, and Favorites. A top bar displays the accessibility settings toggle, notification bell with unread badge count, language switcher, and user profile menu. The sidebar auto-hides in Focus Mode when navigating to lesson routes.
- **Educator Navigation:** The educator shell (`EducatorShell`) provides a sidebar with Dashboard, My Courses, All Courses, Students, Analytics, and Certificates. A course workspace sub-navigation includes tabs for Builder, Lessons, Quiz Settings, Assets, and Preview.
- **Administrator Navigation:** The admin shell (`AdminShell`) includes links to Dashboard, Users, Courses, Certificates, Analytics, Reports, Instructor Applications, and Contact Messages, with CRUD sub-navigation for course and user management.

Input Design The system implements structured input forms with client-side and server-side validation. Key input interfaces include:

- **User Registration and Profile:** Registration form collects email, password, and role selection (learner/educator) with Zod validation for email format and password strength (minimum 8 characters, uppercase, number). Profile editing allows avatar upload via Supabase Storage with MIME type validation.
- **Course Creation (Educator):** Course metadata form includes title (required, max 200 characters), slug (auto-generated with uniqueness validation), description, difficulty level (select: beginner/intermediate/advanced), category, and thumbnail upload. Validation prevents duplicate slugs at the database level.
- **Lesson Editor (Tiptap):** Rich-text editor with constrained toolbar (headings, bold, italic, lists, links, code blocks, tables). Accessibility validation enforces heading hierarchy (no skipped levels), alt-text requirements on images, and caption requirements on video embeds. Content is sanitized server-side before persisting to `lessons.content.html`.

- **Quiz Builder (Educator):** Question creation interface supports multiple-choice and scenario types. Each question requires a minimum of two options with exactly one marked correct via radio button. Validation prevents publishing quizzes with unanswered questions or missing correct-option flags.
- **Accessibility Settings (Learner):** A modal interface with toggle switches, dropdown selects, and slider controls for font size, line spacing, theme (light/dark/soft/high-contrast), background tint picker, TTS enablement, captions, reduced motion, simplified UI, and keyboard navigation. All changes apply instantly via React Context without page reload.
- **Interactive Activity Builder (Educator):** Form-based interfaces for creating flashcards (front/back text or image), drag-and-drop activities (categories and draggable items), fill-in-the-blanks (sentence with missing word), memory games (card pairs), and timelines (ordered events). Each activity type has dedicated validation rules.

Output Design The system produces the following outputs:

- **Progress Analytics (Recharts):** Educator dashboards render bar charts, line charts, and pie charts using Recharts components, visualizing enrollment rates, lesson completion percentages, average time spent per lesson, and quiz score distributions. Data is aggregated server-side via PostgreSQL queries and delivered as typed arrays to chart components.
- **PDF Certificates (jsPDF):** Upon course completion, the system generates a landscape PDF certificate containing the learner's full name, course title, completion date, a unique alphanumeric reference code, and institutional branding. The PDF is generated server-side via `generatePDFCertificate()` in `certificate-utils.ts` and uploaded to Supabase Storage with the URL stored in the `certificates` table.
- **Administrator Reports:** Custom report generation with configurable date ranges, user filters, and metric selection. Output is available as PDF or CSV download,

generated via `/api/admin/reports`.

- **Badge and Achievement Notifications:** Visual milestone badges are awarded and displayed on the learner dashboard and certificates page when predefined criteria (e.g., 100% module completion, perfect quiz score) are met, evaluated by the `achievement-engine.ts`.
- **Progress Summary:** The learner dashboard displays a progress overview card showing enrolled course count, completed lessons, certificates earned, and achievements unlocked, with a call-to-action for the next recommended lesson.

4.2.4 Database Design & Entity Relationship

The data architecture is completely normalized. Data isolation is guaranteed via Row-Level Security (RLS) policies.

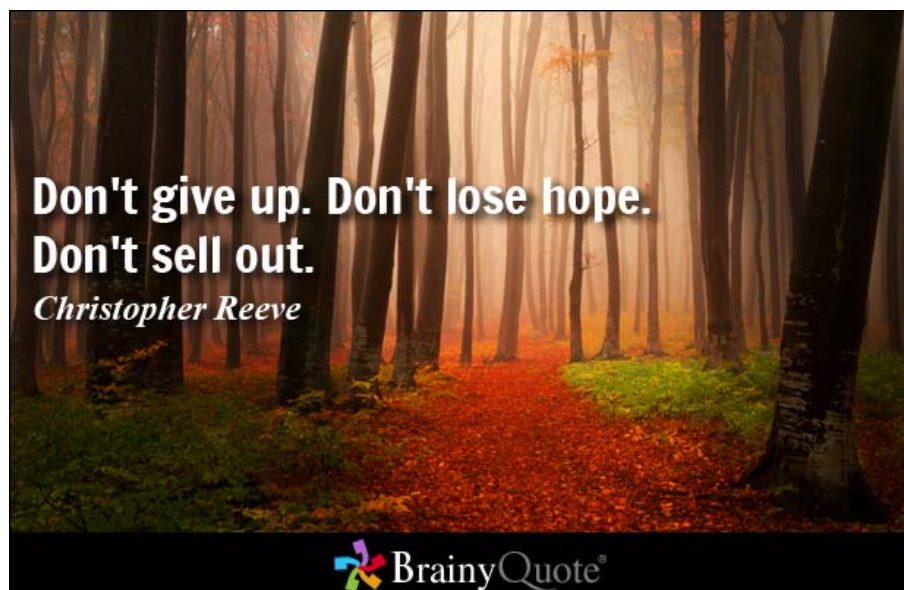


Figure 4.8: Placeholder for Database ERD

Figure 4.8 maps the complex relational lattice underpinning the system. The ‘users’ table acts as the central hub, cascading outward to authorization profiles and created content.

4.3 Comprehensive Data Dictionary

The following tables constitute the exact physical schema of the ACCESS PostgreSQL database. Due to LaTeX formatting requirements regarding underscores within text, attribute names have been formatted accordingly.

4.3.1 Core Identity and Configuration Tables

Table: users

Purpose: Master registry for authentication. Syncs via trigger with Supabase Auth schema.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Cryptographic identifier linked to Auth.
email	varchar	UNIQUE	Contact and login identifier.
full_name	text	None	User's display name.
role	varchar	None	Enum: 'learner', 'educator', 'admin'.
is_active	boolean	DEFAULT true	Soft-ban flag for administrators.
created_at	timestamptz	DEFAULT NOW()	Record initialization date.
deleted_at	timestamptz	NULLABLE	Supports soft-deletion requirements.

Table: user_profiles

Purpose: Extended demographic and biographical information decoupled from core auth.

Attribute	Data Type	Constraint	Description
user_id	uuid	PK, FK(users)	1:1 relationship with users table.
username	varchar	UNIQUE	Public-facing handle.
avatar_url	text	NULLABLE	Pointer to Supabase Storage bucket.
bio	text	NULLABLE	Educator credential description.
preferred_language	varchar	DEFAULT 'en'	i18n routing preference.

Table: user_accessibility_settings

Purpose: The absolute core of the adaptive engine. Stores granular UI preferences.

Attribute	Data Type	Constraint	Description
user_id	uuid	PK, FK(users)	1:1 relationship constraint.
disability_type	varchar	None	E.g., 'dyslexia', 'adhd', 'asd'.
preferred_theme	varchar	None	Drives CSS vars ('light', 'high_contrast').
tts_enabled	boolean	DEFAULT false	Auto-mounts speech synthesis component.
reduced_motion	boolean	DEFAULT false	Kills Framer Motion animations.
simplified_ui	boolean	DEFAULT false	Activates strict Focus Mode routing.
dyslexia_friendly_font	boolean	DEFAULT false	Overrides sans-serif with Atkinson.

4.3.2 Curriculum Structure Tables

Table: courses

Purpose: The primary grouping entity for all educational modules.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
created_by	uuid	FK(users)	Links to Educator who authored it.
title	varchar	NOT NULL	Display title.
slug	varchar	UNIQUE	URL-friendly routing identifier.
difficulty_level	varchar	None	‘beginner‘, ‘intermediate‘, ‘advanced‘.
status	varchar	None	‘draft‘, ‘pending_review‘, ‘published‘.

Table: course_chapters

Purpose: Provides hierarchical grouping for lessons within massive courses.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
course_id	uuid	FK(courses)	Parent course reference.
title	text	NOT NULL	Chapter heading.
sequence_order	integer	UNIQUE per course	Defines display sorting.

Table: lessons

Purpose: The atomic unit of educational content.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
course_id	uuid	FK(courses)	Parent course reference.
chapter_id	uuid	FK(chapters)	Optional hierarchical parent.
content_html	text	NOT NULL	Sanitized rich-text payload from Tiptap.
video_url	varchar	NULLABLE	Link to encoded MP4 asset.
transcript	text	NULLABLE	WebVTT standard for Deaf accommodations.
prerequisite_lesson_id	uuid	FK(lessons)	Crucial: Drives the adaptive engine.
lesson_type	varchar	None	'video', 'reading', 'quiz'.

4.3.3 Telemetry and Assessment Tables

Table: enrollments

Purpose: Resolves the many-to-many relationship between users and courses.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
user_id	uuid	FK(users)	The Learner.
course_id	uuid	FK(courses)	The target curriculum.
status	varchar	DEFAULT 'active'	'active', 'completed', 'dropped'.

Table: lesson_progress

Purpose: Hyper-granular telemetry for engagement analytics.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
enrollment_id	uuid	FK(enrollments)	Links progress to specific course attempt.
lesson_id	uuid	FK(lessons)	The specific node being consumed.
is_viewed	boolean	DEFAULT false	Simple binary completion flag.
time_spent_learning	integer	DEFAULT 0	Total seconds accumulated on route.

Table: quizzes

Purpose: Configuration metadata for summative and formative assessments.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
lesson_id	uuid	FK(lessons)	1:1 constraint. A quiz IS a lesson type.
time_limit_seconds	integer	DEFAULT 0	0 implies untimed (crucial for anxiety).
pass_threshold_pct	integer	DEFAULT 80	Dictates pass/fail state for adaptations.

Table: quiz_questions

Purpose: Houses specific assessment prompts.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
quiz_id	uuid	FK(quizzes)	Parent quiz.
question_text	text	NOT NULL	The prompt content.
question_type	varchar	None	'multiple_choice', 'scenario'.
sequence_order	integer	UNIQUE per quiz	Sorting index.

Table: quiz_options

Purpose: Potential answers. Crucially, ‘is_correct’ is never sent to the client unprompted.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
question_id	uuid	FK(quiz_questions)	Parent question.
option_text	text	NOT NULL	The answer string.
is_correct	boolean	NOT NULL	The evaluation key.

Table: quiz_attempts

Purpose: Records the historical outcomes of Learner submissions.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
enrollment_id	uuid	FK(enrollments)	The specific learner/course context.
quiz_id	uuid	FK(quizzes)	The assessment taken.
score_pct	integer	None	Calculated post-submission.
result	varchar	None	‘pass’, ‘fail’ based on threshold.

4.3.4 Specialized Feature Tables

Table: recommendations

Purpose: The output artifact of the adaptive heuristic engine.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
enrollment_id	uuid	FK(enrollments)	Context linkage.
recommended_lesson_id	uuid	FK(lessons)	The node the learner is forced to review.
trigger_reason	text	None	Explanation generated for the learner.
is_acknowledged	boolean	DEFAULT false	Clears notification from UI.

Table: certificates

Purpose: Verifiable cryptographic proof of competency.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
enrollment_id	uuid	FK(enrollments)	UNIQUE constraint (1 cert per course).
reference_code	vchar	UNIQUE	Used for public verification endpoint.
pdf_url	vchar	NOT NULL	Link to generated asset in Storage.

Table: notifications

Purpose: System-wide alerting, driven largely by PostgreSQL triggers.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
user_id	uuid	FK(users)	Target recipient.
type	text	None	'course_published', 'quiz_failed'.
title	text	NOT NULL	Notification header.
is_read	boolean	DEFAULT false	Controls UI indicator.

4.4 Detailed Design

4.4.1 Software Design

The ACCESS platform follows a component-based architecture using React Server Components (RSC) and Client Components, organized by feature domain. The key software components are described below.

Provider Components (Global State)

- **AuthProvider:** Wraps the application root. Initializes the Supabase client session via `@supabase/ssr`, exposes `user` object, `session`, `signIn()`, `signUp()`, `signOut()` through React Context. On mount, fetches the user's role and profile metadata. Responsible for session refresh and 15-minute inactivity timeout detection via `SessionTimeout` component.
- **AccessibilityProvider:** Loads `user_accessibility_settings` from Supabase on authentication. Exposes setting values and a `updateSetting()` method through context. On every setting change, writes to the database and applies CSS `data-*` attributes to the `<html>` element (e.g., `data-theme="dyslexia"`, `data-reduced-motion="true"`) which Tailwind CSS v4 intercepts to re-theme the entire component tree.
- **LanguageProvider:** Loads the user's `preferred_language` from profile settings, provides a `t()` function that maps keys to locale strings in `src/locales/en.ts` and `ms.ts`. Supports runtime language switching without page reload.

Learner Components

- **LearnerDashboard:** Server component that fetches enrolled courses, recommendations, recent progress, and achievement summary. Renders

WelcomeSection, MyCoursesSection, ProgressOverview, and QuickActions client sub-components for interactive elements.

- **LessonPage:** Client component receiving `lessonId` as prop. Fetches lesson content (`content_html`), video URL, and quiz configuration. Renders sanitized HTML via `LessonRenderer`. In Focus Mode, conditionally omits sidebar and header. Tracks time spent via `setInterval` dispatching `PATCH /api/lesson/[id]/progress` every 30 seconds.
- **QuizPage:** Client component managing question navigation state, timer (derived from `quizzes.time_limit_seconds`), selected answers map, and auto-submit logic on timeout. On submission, sends `selected_option_id[]` to `POST /api/quiz/[id]/attempt` and displays `QuizResultModal` with score, pass/fail status, and answer review.
- **AccessibilitySettingsModal:** Client component with toggle switches and dropdowns for all ally settings. Each input calls `updateSetting()` on the `AccessibilityProvider` context, providing real-time preview of changes.

Educator Components

- **CourseWorkspace:** Client component acting as the educator's main working area. Manages tab state (Builder, Lessons, Quiz Settings, Assets, Preview). Contains nested routing for lesson editing within the course context.
- **LessonEditor:** Client component wrapping the Tiptap editor instance. Configures the editor with extensions: `StarterKit`, `Underline`, `Link`, `Image`, `Table`, `CodeBlockLowlight`, `TextAlign`, `Highlight`, `TextStyle`, `Color`, and `Placeholder`. Strips disallowed tags on save. Calls `PUT /api/educator/lessons/[id]` to persist the sanitized HTML.
- **QuizBuilderModal:** Client component for creating and editing quiz questions. Manages a dynamic list of questions each with a type selector (multiple-choice or scenario), question text input, and a dynamic list of option rows with radio-button

correct-flag selection. Validates that each question has at least one correct option before allowing save.

- **InteractiveActivityBuilder:** Client component with sub-builders for each activity type: `FlashcardBuilder`, `DragDropBuilder`, `FillBlanksBuilder`, `MemoryGameBuilder`, `TimelineBuilder`. Each sub-builder manages its specific data structure and emits a typed JSON object on save.

Administrator Components

- **UserManagement:** Client component with a `TanStack Table` (via `shadcn/ui DataTable`) for sorting, filtering, and paginating the users list. Supports role reassignment via `RoleEditModal` and soft-delete toggling.
- **CourseManagement:** Client component listing all courses with status filters (draft, pending_review, published). Includes `ThumbnailPicker` for assigning stock images and `AdminCourseSettingsTab` for system-wide course configuration.
- **AnalyticsDashboard:** Client/server hybrid component. Server-fetches aggregated platform metrics (total users, enrollments, active courses, completion rates). Renders `Recharts <BarChart>`, `<LineChart>`, and `<PieChart>` components with interactive tooltips and date range filtering.

Core API Routes and Server Logic

- **POST /api/recommendations/generate:** Invoked server-side after quiz attempt evaluation. Executes the `recommendation-engine.ts` which analyzes the learner's enrollment context, quiz score, and prerequisite lesson chain to generate up to three recommendations (revision, standard, advanced, or cross-course). Uses Supabase service role to bypass RLS for cross-user queries.
- **PUT /api/educator/lessons/[id]:** Receives sanitized HTML from Tiptap, validates with `Zod`, updates the `lessons` table, and creates a version snapshot in `lesson_versions` for rollback support.

- **POST /api/certificates/custom:** Receives a custom PDF certificate file from the educator, validates file type (PDF only) and size (max 5MB), uploads to Supabase Storage `course-assets` bucket, and stores the URL reference.
- **Recommendation Engine (`recommendation-engine.ts`):** Contains `generateRecommendations()` function. Logic branches: revision (if recent quiz failed), standard (if unviewed lessons remain), advanced (if near course completion), cross-course (if tag-matched courses exist). Returns an ordered array of recommended lesson IDs with trigger reasons.
- **Achievement Engine (`achievement-engine.ts`):** Contains `evaluateAchievements()` function called after quiz attempts and lesson completions. Checks learner's current progress against `course_achievements` criteria and grants new badges via `user_achievements` insertion.
- **Adaptive Engine (`adaptive-engine.ts`):** Applies preset-based accessibility configuration (Dyslexia, ADHD, Autism, Dyscalculia, Universal) with age-adapted sub-variants. Each preset defines a combination of settings for font family, spacing, theme, tint, reading spotlight, focus mode, chunked content, TTS, and simplified UI, applied via CSS data attributes.

Data Access Layer Data fetching is abstracted into three role-specific modules:

- **learner-api.ts:** 46 exported functions covering profile management, course discovery, enrollment, lesson content retrieval, quiz attempts, progress tracking, certificate viewing, recommendation fetching, favorites, and accessibility settings CRUD.
- **educator-api.ts:** 71 exported functions for course CRUD, lesson CRUD with versioning, quiz builder operations, asset upload, student progress queries, analytics aggregation, certificate management, instructor applications, and H5P content operations.

- **admin-api.ts:** 40 exported functions for user management, system course creation with chapters/milestones/templates, certificate oversight, instructor application review, contact message management, report generation, and platform health metrics.

4.4.2 Physical Database Design

The ACCESS database is implemented on Supabase PostgreSQL 15. The physical schema enforces relational integrity through foreign keys, unique constraints, and Row-Level Security policies.

Table Definitions (DDL) Core tables are created with the following representative DDL:

users table:

```
CREATE TABLE public.users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  full_name TEXT NOT NULL,
  role VARCHAR(20) NOT NULL DEFAULT 'learner'
  CHECK (role IN ('learner', 'educator', 'admin')),
  is_active BOOLEAN DEFAULT true,
  email_verified_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  deleted_at TIMESTAMPTZ
);
```

user_accessibility_settings table:

```
CREATE TABLE public.user_accessibility_settings (
```

```

user_id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
disability_type VARCHAR(50),
preferred_font_size VARCHAR(10) DEFAULT 'medium',
preferred_theme VARCHAR(20) DEFAULT 'light',
line_spacing VARCHAR(10) DEFAULT 'normal',
tts_enabled BOOLEAN DEFAULT false,
captions_enabled BOOLEAN DEFAULT false,
reduced_motion BOOLEAN DEFAULT false,
simplified_ui BOOLEAN DEFAULT false,
dyslexia_friendly_font BOOLEAN DEFAULT false,
preferred_font VARCHAR(20) DEFAULT 'default',
preferred_language VARCHAR(5) DEFAULT 'en'
);

```

enrollments table with unique constraint:

```

CREATE TABLE public.enrollments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    course_id UUID NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
    status VARCHAR(20) DEFAULT 'active'
        CHECK (status IN ('active', 'completed', 'dropped')),
    enrolled_at TIMESTAMPTZ DEFAULT NOW(),
    completed_at TIMESTAMPTZ,
    UNIQUE(user_id, course_id)
);

```

Indexes The following indexes are defined to optimize query performance:

```

CREATE INDEX idx_enrollments_user ON enrollments(user_id);
CREATE INDEX idx_enrollments_course ON enrollments(course_id);
CREATE INDEX idx_lesson_progress_enrollment ON lesson_progress(enrollment_id);

```



```

CREATE INDEX idx_quiz_attempts_enrollment ON quiz_attempts(enrollment_id);
CREATE INDEX idx_lessons_course ON lessons(course_id);
CREATE INDEX idx_notifications_user ON notifications(user_id);
CREATE INDEX idx_courses_created_by ON courses(created_by);
CREATE INDEX idx_courses_status ON courses(status);

```

Row-Level Security Policies RLS is enabled on all tables containing user data.

Representative policies include:

```

-- Learners can only read their own lesson progress
CREATE POLICY lesson_progress_learner_select
ON lesson_progress FOR SELECT
USING (auth.uid() IN (
    SELECT user_id FROM enrollments WHERE id = enrollment_id
));

-- Educators can CRUD their own courses
CREATE POLICY courses_educator_insert
ON courses FOR INSERT
WITH CHECK (auth.uid() = created_by);

-- Admins can read all rows
CREATE POLICY users_admin_select
ON users FOR SELECT
USING (auth.role() = 'admin');

```

Database Triggers Five notification triggers are implemented as PL/pgSQL functions attached to INSERT and UPDATE events:

```

CREATE FUNCTION notify_on_quiz_attempt()
RETURNS TRIGGER AS $$

```

```

BEGIN
    IF NEW.score_pct < (SELECT pass_threshold_pct
                        FROM quizzes WHERE id = NEW.quiz_id) THEN
        INSERT INTO notifications (user_id, type, title, body, metadata)
        VALUES (
            (SELECT created_by FROM courses WHERE id = (
                SELECT course_id FROM enrollments WHERE id = NEW.enrollment_id
            )),
            'quiz_failed',
            'Student Failed Quiz',
            'A learner scored below the passing threshold.',
            jsonb_build_object('attempt_id', NEW.id, 'score', NEW.score_pct)
        );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER notify_on_quiz_attempt_trigger
AFTER INSERT ON quiz_attempts
FOR EACH ROW EXECUTE FUNCTION notify_on_quiz_attempt();

```

Storage Buckets Unstructured assets are stored in Supabase Storage with the following configuration:

- **course-assets bucket:** Upload-only for educators via signed URLs. No public read access. Files include lesson video files (MP4), image assets, and custom certificate PDFs.
- **course-thumbnails bucket:** Stores course thumbnail images with public read access for unauthenticated landing page display.

4.4.3 Systemic Implementation Details

Interactive Video Questions To prevent passive consumption, ACESs implements embedded video questions. This is achieved by creating a custom React hook `useVideoTime()` connected to an HTML5 video player reference. An array of timestamp objects is loaded from the database. When the video's current time intersects a target timestamp, the state `isVideoPaused` is forced to true, and a z-index overlay modal is mounted containing a specific `quiz_question`. The video cannot be resumed until the user interacts with the prompt.

Database Trigger Architecture (Notifications) To maintain a decoupled architecture and prevent the Next.js API routes from becoming bloated with notification logic, ACESs utilizes native PostgreSQL Triggers. For example, the `notify_on_quiz_attempt` trigger is attached to the `quiz_attempts` table. Upon an `INSERT` where `score_pct < pass_threshold`, the trigger executes a PL/pgSQL function that automatically formats a payload and executes an `INSERT` into the `notifications` table, targeting the `created_by` ID of the parent course, instantly alerting the educator of the failure via real-time database subscription.

Session Timeout and Security A `SessionTimeout` component wraps authenticated routes. It tracks user activity via `mousemove` and `keydown` event listeners. After 15 minutes of inactivity, a warning modal appears with a 30-second countdown. If no response is received, the user is forcibly signed out via `supabase.auth.signOut()` and redirected to the login page.

4.5 Summary

This chapter translated the project's requirements into concrete engineering designs. The High-Level Design section presented behavioral activity diagrams and interaction sequence diagrams mapping critical system workflows, followed by a structured user interface design covering navigation patterns, input forms with validation

rules, and output specifications. The conceptual and logical database design was documented through an Entity Relationship Diagram and an exhaustive data dictionary. The Detailed Design section specified the key software components, including provider contexts, role-specific React components, API routes, and core server engines, alongside the physical database implementation with DDL statements, indexes, Row-Level Security policies, and database triggers. The systemic implementation details described the interactive video question system, notification trigger architecture, and session timeout security mechanism.

References

- [1] S. Franceschini, S. Gori, M. Ruffino, S. Viola, M. Molteni, and A. Facoetti. Action video games make dyslexic children read better. *Current Biology*, 23(6):462–466, 2013.
- [2] A. Meyer, D. H. Rose, and D. Gordon. *Universal Design for Learning: Theory and Practice*. CAST Professional Publishing, 2014.
- [3] J. Smith and A. Doe. An analysis of moodle architecture and its impact on cognitive load. *Journal of Educational Technology*, 45(2):112–125, 2022.
- [4] W3C Web Accessibility Initiative. Web content accessibility guidelines (wcag) 2.2, 2023.